Phase II Project Report
RetailHero Uplift Modeling: CCP vs CCU Evaluation

Team Submission

29 April 2026

Course: HSL-530 Data Analytics for Business & Society

Team Members: Anand Deep (21322003), Aniket (21322005), Madhav Mukund (21322018), Param Mahabare (21322021)

Abstract

This report documents the Phase II implementation of a value-aware uplift-modeling pipeline for the RetailHero / X5 retail intervention dataset. The work extends Phase I, which established that the campaign was effectively randomized, that the average treatment effect was modest, and that response heterogeneity was large enough to justify uplift modeling. Phase II operationalizes that finding by comparing a standard customer purchase-propensity baseline (CCP) against three customer churn uplift (CCU) estimators under one reproducible evaluation framework. The pipeline rebuilds features directly from the raw CSV files, trains all models on the full extracted dataset, evaluates them on a stratified holdout set, and reports Qini, liftup, predictive maximum profit, and maximum profit uplift. The final result is that the uplift random forest is the strongest incremental-targeting model, outperforming the CCP baseline on uplift-specific metrics while preserving a transparent, reproducible workflow.

Contents

1 Project Objective **3**

2 Data and Verified Phase I Findings **3**
- 2.1 Dataset . . . 3
- 2.2 Phase I Findings Revalidated from Raw Data . . . 3
- 2.3 Why These Findings Justify Phase II . . . 4

3 Phase II Methodology **4**
- 3.1 Design Principles . . . 4
- 3.2 Feature Engineering . . . 4
- 3.3 Data Split . . . 5
- 3.4 Models . . . 5
  - 3.4.1 Model A: CCP baseline . . . 5
  - 3.4.2 Model B1: Two-model logistic uplift . . . 5
  - 3.4.3 Model B2: Lo-style interaction logistic uplift . . . 5
  - 3.4.4 Model B3: Uplift random forest . . . 5
- 3.5 Evaluation Framework . . . 6

4 Implementation Summary **6**
4.1 Code Organization . . . 6
4.2 Implementation Integrity Checks . . . 6

5 Results **7**
5.1 Selected Hyperparameters . . . 7
5.2 Validation Summary . . . 7
5.3 Interpretation . . . 7
5.4 Figures . . . 8

6 Defense of the Work **9**
6.1 Why the pipeline is methodologically sound . . . 9
6.2 Why the model comparison is fair . . . 9
6.3 Why the conclusions are credible . . . 10

7 Limitations **10**

8 Conclusion **10**

A Relevant Code Listings **11**
A.1 Client and Product Feature Construction . . . 11
A.2 Uplift Model Wrappers . . . 14
A.3 Pipeline Orchestration . . . 16

B Reproduction Commands **19**

1 Project Objective

The objective of Phase II was to implement a defensible, reproducible comparison between:

- **Model A: CCP baseline**, a standard purchase-propensity model trained only on the control group.
- **Model B: CCU models**, uplift estimators that rank customers by expected incremental response to treatment.

The practical research question is:

> Can the intervention be targeted more efficiently by ranking customers on expected incremental value rather than raw purchase likelihood?

This framing matters because a customer who is highly likely to buy anyway may still be a poor target for campaign spend. Uplift modeling is valuable only if it changes that ranking in a measurable and profitable way.

2 Data and Verified Phase I Findings

2.1 Dataset

The project uses the RetailHero uplift dataset with the following raw inputs:

- `clients.csv`
- `products.csv`
- `purchases.csv`
- `uplift_train.csv`
- `uplift_test.csv`
- `uplift_sample_submission.csv`

The purchases file is the dominant data asset and contains approximately 4.4 GB of transaction history. Because of its size, the implementation uses DuckDB-backed SQL aggregation rather than loading the entire file into a single in-memory pandas frame.

2.2 Phase I Findings Revalidated from Raw Data

Phase I findings were rechecked directly from raw data via the verification script rather than relying on notebook narrative alone. The key findings are summarized in Table 1.

Table 1: Verified Phase I findings used to motivate Phase II

Finding	**Verified value**
Treatment / control ratio	0.999230 (99,981 treated / 100,058 control)
Baseline uplift	3.323 percentage points
Average uplift in low-frequency deciles 0–3	4.452 percentage points
Average uplift in high-frequency deciles 6–9	1.755 percentage points
Sure Things share	30.176% of training clients
Age anomalies	min = -7,491, max = 1,901, flagged = 1,145
Distinct non-null `level_1` categories	3
Test-set clients with purchase history	200,123 / 200,123

2.3 Why These Findings Justify Phase II

These findings defend the Phase II design in three ways:

1. The near-perfect treatment/control balance means uplift estimates can be interpreted causally without heavy post-hoc adjustment.
2. The modest average treatment effect means a single global response model is likely to be wasteful because the mean hides heterogeneity.
3. The concentration of uplift among lower-frequency customers means targeting by purchase probability alone is not aligned with incremental value.

3 Phase II Methodology

3.1 Design Principles

Phase II was built around four design principles:

1. **Raw-data-first reproducibility**: all Phase II features are rebuilt from `data/*.csv` rather than notebook-only exports.
2. **Comparable feature space**: all models consume the same deterministic numeric feature matrix.
3. **Value-aware evaluation**: uplift models are not judged only by response prediction, but by incremental targeting value.
4. **Transparent implementation**: the pipeline is organized into feature, model, metric, and orchestration modules rather than one monolithic script.

3.2 Feature Engineering

The final Phase II matrix contains 44 numeric features with zero missing values after deterministic filling and explicit missingness indicators. The feature groups are:

- **Demographics**: `age`, `age_clean`, `age_flagged`, gender indicators, `gender_known`
- **Redemption behavior**: redeemed flag, time-to-redeem, card tenure, and redemption recency
- **Purchase behavior**: spend, transactions, trips, items, basket metrics, stores, products, points, recency, tenure
- **Product behavior**: alcohol spend share, own-trademark spend share, netto aggregates, spend-share across the three `level_1` groups, and brand concentration (HHI)

This design is defensible because it follows directly from Phase I. The hashed category hierarchy is not semantically interpretable, so the implementation does not pretend it is. Instead, it transforms product metadata into client-level behavioral features that can still carry signal.

3.3 Data Split

The Phase II pipeline uses:

- an outer holdout split of 20% of `uplift_train` for validation,
- stratification on the joint label `(treatment_flg, target)`,
- a fixed random seed of 123,
- a smaller inner tuning split inside the training fold only.

This is the correct compromise for the current stage of the project. It preserves a clean validation set while keeping the implementation simple enough to inspect and rerun.

3.4 Models

3.4.1 Model A: CCP baseline

The CCP baseline is an `XGBClassifier` trained on control-group rows only. It predicts purchase propensity rather than treatment effect.

This design is defensible because:

- it creates a strong conventional benchmark,
- it avoids learning treatment-induced behavior when estimating organic purchase likelihood,
- it is exactly the sort of model a business team might deploy if uplift were ignored.

3.4.2 Model B1: Two-model logistic uplift

Two separate regularized logistic models are trained, one on treated customers and one on control customers. The uplift score is:

$$\hat{u}(x) = \hat{P}(Y = 1 \mid T = 1, X = x) - \hat{P}(Y = 1 \mid T = 0, X = x)$$

This is the most literal implementation of the roadmap formula.

3.4.3 Model B2: Lo-style interaction logistic uplift

A single logistic model is trained on all rows with:

- the treatment indicator as a feature,
- first-order interactions between treatment and all covariates.

The uplift score is obtained by scoring each customer twice, once under treatment and once under control, then taking the difference.

This is defensible because it provides a structured single-model alternative to the two-model approach.

3.4.4 Model B3: Uplift random forest

The non-linear uplift model is implemented using `causalml`'s `UpliftRandomForestClassifier` with:

- `KL` split evaluation,
- string-encoded treatment labels required by the package API,
- light tuning over tree count, feature count, depth, leaf size, and regularization.

This is defensible because it is the most expressive model in the comparison and can capture non-linear treatment heterogeneity that linear models cannot.

3.5 Evaluation Framework

Phase II evaluates the models with four complementary views:

- **Qini curves**: how well the model ranks incremental responders
- **Liftup curves**: uplift-specific analogues of lift behavior
- **Predictive maximum profit (MP)**: used for the CCP benchmark
- **Maximum profit uplift (MPU)**: used for incremental targeting

The default economics are:

- average basket value = empirical average trip value = 4425.003 RUB
- SMS cost = 1 RUB

Sensitivity analysis then varies both assumptions.

4 Implementation Summary

4.1 Code Organization

The Phase II codebase is modular:

- `phase2_features.py`: raw-data feature construction and cache writing
- `phase2_models.py`: model wrappers, tuning, and scoring
- `phase2_metrics.py`: ranking, Qini, liftup, MP, and MPU calculations
- `phase2_pipeline.py`: full orchestration and artifact generation
- `verify_phase2_outputs.py`: artifact and consistency checks

4.2 Implementation Integrity Checks

The completed pipeline passed all current verification checks:

- 200,039 Phase II train rows and 200,123 Phase II test rows
- 44 model features
- zero missing feature values in both train and test matrices
- valid category-share algebra with maximum absolute error 1.343×10^{-14}
- finite validation and test scores for all four models
- valid target-share bounds for Qini, liftup, and sensitivity outputs

This matters because a submission-ready report should not only present model outcomes; it should demonstrate that the pipeline itself is internally coherent.

5 Results

5.1 Selected Hyperparameters

The final selected hyperparameters from the current full run are shown in Table 2.

Table 2: Selected model hyperparameters

Model	Selected parameters
Model A - CCP baseline	`n_estimators=250`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.85`, `colsample_bytree=0.85`
Model B1 - Two-model logistic	`C=1.0`
Model B2 - Lo interaction logistic	`C=10.0`
Model B3 - Uplift random forest	`n_estimators=25`, `max_features=12`, `max_depth=5`, `min_samples_leaf=100`, `min_samples_treatment=50`, `n_reg=10`

5.2 Validation Summary

The full validation summary is shown in Table 3. The validation holdout contains 40,008 rows.

Table 3: Phase II validation summary

Model	Qini AUC	Top-decile liftup	MPU	Optimal target share
Model B3 - CCU Uplift Random Forest	0.004759	3.637315	158.307694	0.7475
Model B1 - CCU Two-Model Logistic	0.003297	3.239985	146.083912	1.0000
Model B2 - CCU Lo Interaction Logistic	0.003245	3.231146	146.083912	1.0000
Model A - CCP Baseline	-0.002239	0.016284	146.083912	1.0000

For completeness, the CCP baseline also achieved a predictive maximum profit of 2742.063982 under the default economics. That figure is intentionally reported separately from MPU because it is not the same business question. Predictive MP rewards finding likely buyers. MPU rewards finding customers whose behavior is changed by treatment. Those are related but not interchangeable objectives.

5.3 Interpretation

Three findings matter most:

1. The uplift random forest is the strongest CCU model on both Qini and MPU.
2. The logistic uplift models show positive uplift discrimination and materially outperform the CCP baseline on top-decile liftup.
3. The CCP baseline is excellent at ranking likely buyers, but poor at isolating persuadable buyers near the top of the list.

That result is exactly what Phase I suggested would happen. High-frequency customers are often sure things. A strong purchase-propensity model can rank them very highly while still producing weak incremental targeting value. The uplift models reverse that logic by ranking customers on expected treatment effect.

5.4 Figures

Figure 1 shows the Qini curves. The uplift random forest dominates the uplift-specific ranking metric, while the CCP baseline is negative relative to random on this criterion.

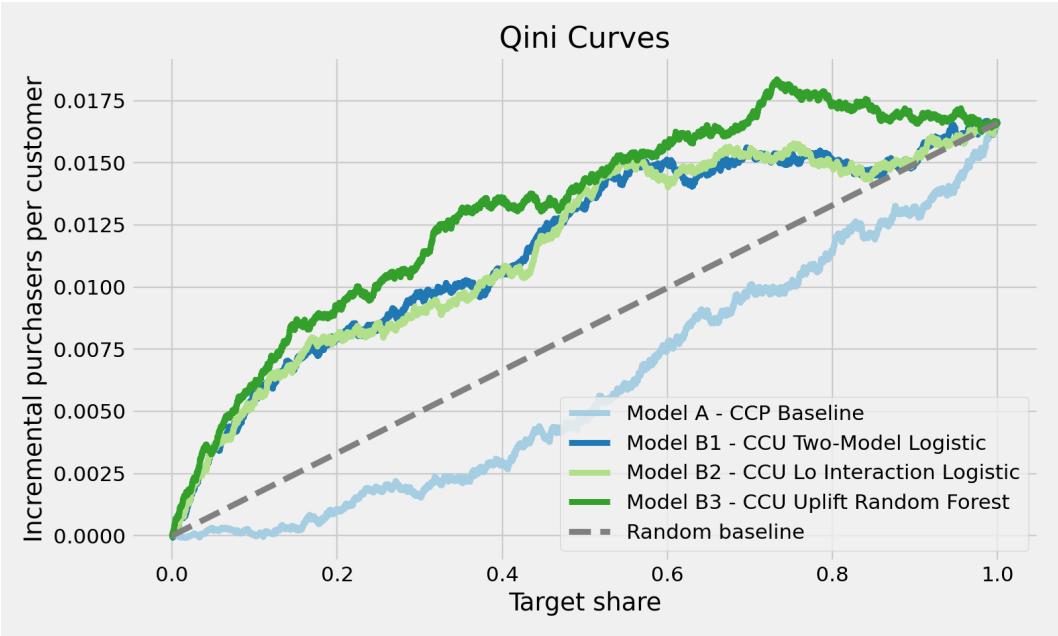

Figure 1: Qini curves on the Phase II validation holdout.

Figure 2 shows the liftup curves. The CCP baseline has very high top-decile response lift but almost no top-decile liftup, which is precisely the failure mode Phase I warned about.

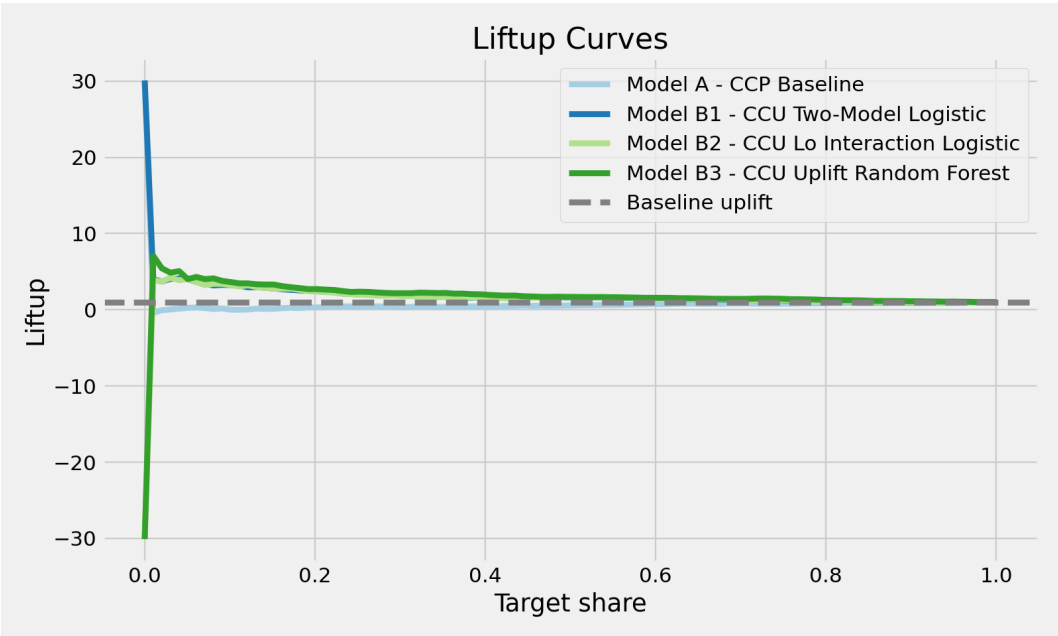

Figure 2: Liftup curves on the Phase II validation holdout.

Figure 3 compares incremental profit curves and the CCP predictive MP curve. The uplift random forest is the only model that clearly pushes the incremental-profit frontier upward.

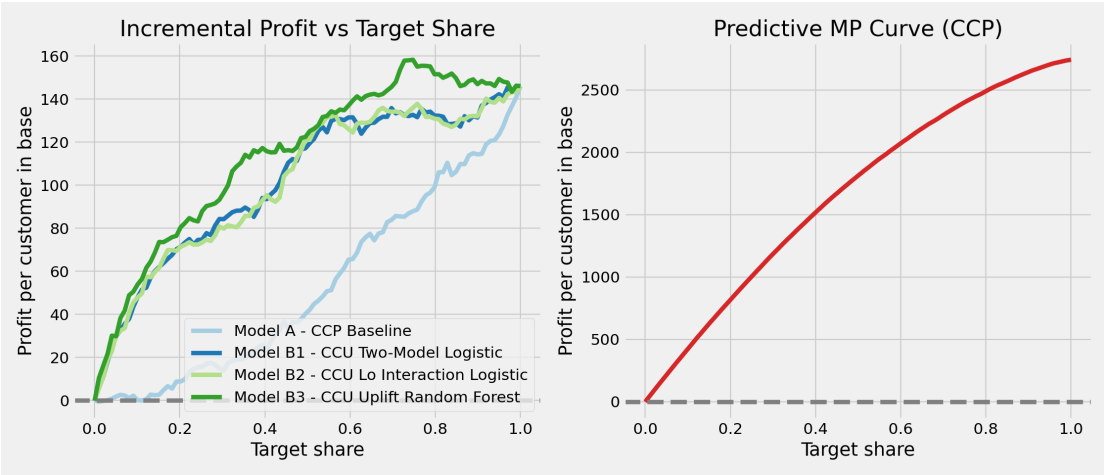

Figure 3: Incremental profit and CCP predictive MP curves.

Figure 4 shows that the relative ordering is not a single-point artifact of the chosen economics; the uplift random forest remains competitive under the current sensitivity sweep.

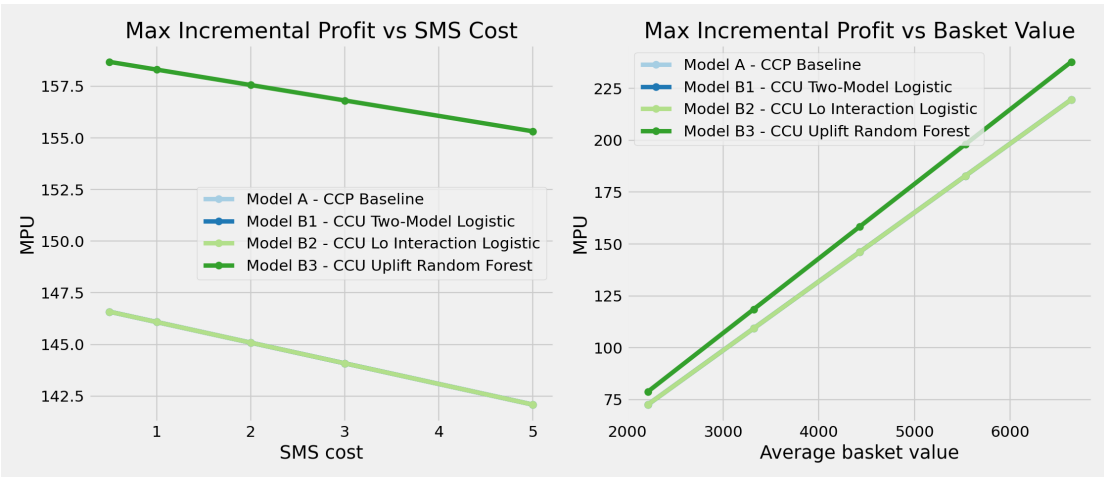

Figure 4: Sensitivity of maximum incremental profit to basket value and SMS cost assumptions.

6 Defense of the Work

6.1 Why the pipeline is methodologically sound

The work is defensible because it avoids three common errors:

1. **It does not confuse purchase likelihood with incremental value.**
2. **It does not rely on notebook-only, hand-built outputs.**
3. **It does not treat anonymized product categories as if they were interpretable business taxonomies.**

Instead, the implementation follows the empirical evidence established in Phase I and uses a validation framework aligned with the targeting objective.

6.2 Why the model comparison is fair

The comparison is fair because:

- all models use the same raw source data,
- all models use the same engineered feature base,
- all uplift models are tuned on inner splits only,
- all final metrics come from the same outer validation set.

6.3 Why the conclusions are credible

The conclusions are credible because they are supported simultaneously by:

- Phase I causal balance checks,
- full-run validation outputs,
- uplift-specific ranking curves,
- profit-oriented summaries,
- formal verification scripts for both feature integrity and saved artifacts.

7 Limitations

The current work still has limits:

- The hashed product hierarchy limits semantic interpretation.
- The current profit analysis depends on default economic assumptions, though sensitivity analysis reduces that risk.
- The uplift random forest uses a heavier third-party package and is slower than the logistic baselines.
- The implementation is an experimental analysis pipeline, not a production scoring service.

These limitations are acceptable at this stage because the objective of Phase II was comparative methodology and defensible empirical evidence, not deployment.

8 Conclusion

Phase II successfully translated the Phase I findings into a reproducible uplift-modeling comparison. The resulting pipeline demonstrates that:

- a conventional CCP purchase-propensity ranking is not sufficient for incremental targeting,
- uplift-specific models are empirically justified on this dataset,
- the uplift random forest is currently the strongest model under the implemented evaluation framework.

The project therefore supports the central claim that targeting customers by expected incremental response is more defensible than targeting them by unconditional purchase probability alone.

A Relevant Code Listings

The report includes the most relevant implementation excerpts directly from the working codebase.

A.1 Client and Product Feature Construction

Listing 1: Client-level feature construction in `phase2_features.py`

```
def _read_clients(clients_path: str | Path, campaign_date: pd.Timestamp) -> pd.
    DataFrame:
    clients = pd.read_csv(
        clients_path,
        parse_dates=["first_issue_date", "first_redeem_date"],
    )

    clients["age_flagged"] = ((clients["age"] < 0) | (clients["age"] > 100)).
        astype(int)
    clients["age_clean"] = clients["age"].clip(lower=0, upper=100)

    clients["gender_M"] = (clients["gender"] == "M").astype(int)
    clients["gender_F"] = (clients["gender"] == "F").astype(int)
    clients["gender_U"] = (clients["gender"] == "U").astype(int)
    clients["gender_known"] = (clients["gender"] != "U").astype(int)

    clients["ever_redeemed"] = clients["first_redeem_date"].notna().astype(int)
    clients["days_to_redeem"] = (
        clients["first_redeem_date"] - clients["first_issue_date"]
    ).dt.days
    clients["days_to_redeem_missing"] = clients["days_to_redeem"].isna().astype
        (int)
    clients["days_to_redeem"] = clients["days_to_redeem"].fillna(-1)

    clients["issue_age_days"] = (campaign_date - clients["first_issue_date"]).
        dt.days
    clients["redeem_recency_days"] = (
        campaign_date - clients["first_redeem_date"]
    ).dt.days
    clients["redeem_recency_missing"] = clients["redeem_recency_days"].isna().
        astype(int)
    clients["redeem_recency_days"] = clients["redeem_recency_days"].fillna(-1)

    return clients[
        [
            "client_id",
            "age",
            "age_clean",
            "age_flagged",
            "gender_M",
            "gender_F",
            "gender_U",
            "gender_known",
            "ever_redeemed",
            "days_to_redeem",
            "days_to_redeem_missing",
            "issue_age_days",
            "redeem_recency_days",
            "redeem_recency_missing",
        ]
    ].copy()
```

Listing 2: Product-behavior aggregation and share-feature construction in `phase2_features.py`

```python
def _build_product_behavior_features(
    purchases_path: str | Path,
    products_path: str | Path,
    product_meta: dict[str, Any],
) -> pd.DataFrame:
    netto_p99 = product_meta["netto_p99"]
    level1_categories = product_meta["level1_categories"]

    category_spend_cols = ",\n".join(
        [
            f"    sum(case when level_1 = '{category}' then purchase_sum else 0
                end) "
            f"as level1_spend_{category}"
            for category in level1_categories
        ]
    )

    conn = _connect()
    try:
        query = f"""
        with products_clean as (
          select
            product_id,
            level_1,
            brand_id,
            coalesce(is_alcohol, 0) as is_alcohol,
            coalesce(is_own_trademark, 0) as is_own_trademark,
            case
              when netto is null then null
              else least(netto, {netto_p99})
            end as netto_clean
          from read_csv(?, header=true, columns={PRODUCT_COLUMNS_SQL})
        ),
        joined as (
          select
            purchases.client_id,
            coalesce(purchases.purchase_sum, 0) as purchase_sum,
            coalesce(purchases.product_quantity, 0) as product_quantity,
            products_clean.level_1,
            products_clean.brand_id,
            products_clean.is_alcohol,
            products_clean.is_own_trademark,
            products_clean.netto_clean
          from read_csv(?, header=true, columns={PURCHASE_COLUMNS_SQL})
              purchases
          left join products_clean
          using (product_id)
        ),
        client_rollup as (
          select
            client_id,
            sum(purchase_sum) as product_join_total_spend,
            sum(case when is_alcohol = 1 then purchase_sum else 0 end) as
                alcohol_spend,
            sum(case when is_own_trademark = 1 then purchase_sum else 0 end) as
                own_trademark_spend,
            sum(case when level_1 is null then purchase_sum else 0 end) as
                unknown_level1_spend,
            count(distinct brand_id) as n_brands,
            sum(coalesce(netto_clean, 0) * product_quantity) as netto_total,
            sum(case when netto_clean is null then product_quantity else 0 end)
                as missing_netto_quantity,
```

```
{category_spend_cols}
          from joined
          group by client_id
        ),
        brand_spend as (
          select
            client_id,
            brand_id,
            sum(purchase_sum) as brand_spend
          from joined
          where brand_id is not null
          group by client_id, brand_id
        ),
        brand_hhi as (
          select
            brand_spend.client_id,
            sum(power(brand_spend.brand_spend / nullif(client_rollup.
                product_join_total_spend, 0), 2)) as brand_hhi
          from brand_spend
          inner join client_rollup
            on brand_spend.client_id = client_rollup.client_id
          group by brand_spend.client_id
        )
        select
          client_rollup.*,
          coalesce(brand_hhi.brand_hhi, 0) as brand_hhi
        from client_rollup
        left join brand_hhi
          on client_rollup.client_id = brand_hhi.client_id
        order by client_rollup.client_id
        """
        features = conn.execute(
            query,
            [
                _normalize_path(products_path),
                _normalize_path(purchases_path),
            ],
        ).df()
    finally:
        conn.close()

    return features


def _add_share_features(df: pd.DataFrame, level1_categories: list[str]) -> pd.
    DataFrame:
    total_spend = df["total_spend"].replace(0, np.nan)

    df["alcohol_spend_share"] = (df["alcohol_spend"] / total_spend).fillna(0)
    df["own_trademark_spend_share"] = (df["own_trademark_spend"] / total_spend)
        .fillna(0)
    df["unknown_level1_spend_share"] = (df["unknown_level1_spend"] /
        total_spend).fillna(0)

    for category in level1_categories:
        spend_col = f"level1_spend_{category}"
        share_col = f"level1_share_{category}"
        df[share_col] = (df[spend_col] / total_spend).fillna(0)

    item_denom = df["n_items"].replace(0, np.nan)
    df["avg_netto_per_item"] = (df["netto_total"] / item_denom).fillna(0)
    df["missing_netto_share"] = (df["missing_netto_quantity"] / item_denom).
        fillna(0)
```

```
    return df
```

A.2 Uplift Model Wrappers

Listing 3: Two-model logistic, interaction logistic, and uplift random forest wrappers in `phase2_models.py`

```
class TwoModelLogisticUplift:
    def __init__(self, C: float, random_state: int, max_iter: int = 1000) ->
        None:
        self.C = C
        self.random_state = random_state
        self.max_iter = max_iter
        self.treatment_model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )
        self.control_model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "
        TwoModelLogisticUplift":
        self.treatment_model.fit(X[treatment == 1], y[treatment == 1])
        self.control_model.fit(X[treatment == 0], y[treatment == 0])
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        treated_scores = self.treatment_model.predict_proba(X)[:, 1]
        control_scores = self.control_model.predict_proba(X)[:, 1]
        return treated_scores - control_scores


class LoInteractionLogisticUplift:
    def __init__(self, C: float, random_state: int, max_iter: int = 1000) ->
        None:
        self.C = C
        self.random_state = random_state
        self.max_iter = max_iter
        self.model = Pipeline(
            [
```

```
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )

    @staticmethod
    def _design_matrix(X: np.ndarray, treatment: np.ndarray) -> np.ndarray:
        treatment = treatment.reshape(-1, 1).astype(float)
        return np.hstack([X, treatment, X * treatment])

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "
        LoInteractionLogisticUplift":
        self.model.fit(self._design_matrix(X, treatment), y)
        return self

    def predict_counterfactual(self, X: np.ndarray, treatment_value: int) -> np
        .ndarray:
        treatment = np.full(X.shape[0], treatment_value, dtype=int)
        return self.model.predict_proba(self._design_matrix(X, treatment))[:,
            1]

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.predict_counterfactual(X, 1) - self.predict_counterfactual(
            X, 0)


class ControlOnlyXGBModel:
    def __init__(self, random_state: int, **params: Any) -> None:
        self.random_state = random_state
        self.params = params
        self.model = XGBClassifier(
            objective="binary:logistic",
            eval_metric="logloss",
            random_state=self.random_state,
            n_jobs=-1,
            tree_method="hist",
            verbosity=0,
            **self.params,
        )

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "
        ControlOnlyXGBModel":
        control_mask = treatment == 0
        self.model.fit(X[control_mask], y[control_mask])
        return self

    def predict_score(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict_proba(X)[:, 1]


class CausalMLUpliftRandomForest:
    def __init__(self, random_state: int, **params: Any) -> None:
        self.random_state = random_state
        self.params = params
        self.model = UpliftRandomForestClassifier(
```

```
            control_name="control",
            random_state=self.random_state,
            n_jobs=-1,
            evaluationFunction="KL",
            **self.params,
        )

    @staticmethod
    def _encode_treatment(treatment: np.ndarray) -> np.ndarray:
        return np.where(treatment == 1, "treatment", "control")

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "
        CausalMLUpliftRandomForest":
        self.model.fit(
            X.astype(np.float32),
            self._encode_treatment(treatment),
            y.astype(int),
        )
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict(X.astype(np.float32)).reshape(-1)
```

A.3 Pipeline Orchestration

Listing 4: Main orchestration flow in `phase2_pipeline.py`

```
def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)

    bundle = build_phase2_feature_bundle(
        data_dir=args.data_dir,
        output_dir=args.output_dir,
        campaign_date=CAMPAIGN_DATE,
        reuse_cache=args.reuse_features,
    )
    feature_checks = validate_feature_bundle(bundle)

    avg_basket_value = (
        args.avg_basket_value
        if args.avg_basket_value is not None
        else bundle.metadata["avg_basket_value_empirical"]
    )

    outer_train, outer_valid = split_train_validation(
        bundle.train,
        seed=args.random_seed,
        validation_size=0.20,
    )

    model_results = [
        train_ccp_xgboost(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_two_model_logistic(
```

```
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_lo_logistic(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_uplift_random_forest(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
    ]

    scored_validation = outer_valid[["client_id", "treatment_flg", "target"]].
        copy()
    scored_test = bundle.test[["client_id"]].copy()

    metric_rows: list[dict[str, object]] = []
    qini_frames: list[pd.DataFrame] = []
    curve_frames: list[pd.DataFrame] = []
    params_payload = {
        "run_config": {
            "data_dir": str(args.data_dir.resolve()),
            "output_dir": str(args.output_dir.resolve()),
            "random_seed": args.random_seed,
            "sms_cost": args.sms_cost,
            "avg_basket_value": avg_basket_value,
            "reuse_features": args.reuse_features,
            "skip_sensitivity_sweep": args.skip_sensitivity_sweep,
        },
        "feature_checks": feature_checks,
        "models": {},
    }

    for result in model_results:
        score_column = result.model_name.lower().replace(" ", "_").replace("-",
            "").replace("__", "_") + "_score"
        rank_column = score_column.replace("_score", "_rank")

        scored_validation[score_column] = result.validation_scores
        scored_validation[rank_column] = (
            scored_validation[score_column].rank(ascending=False, method="first
                ").astype(int)
        )

        scored_test[score_column] = result.test_scores
        scored_test[rank_column] = scored_test[score_column].rank(ascending=
            False, method="first").astype(int)
```

```
        evaluation = evaluate_ranking(
            y_true=outer_valid["target"].to_numpy(dtype=int),
            treatment=outer_valid["treatment_flg"].to_numpy(dtype=int),
            scores=result.validation_scores,
            avg_basket_value=avg_basket_value,
            sms_cost=args.sms_cost,
        )

        summary_values = dict(evaluation.summary)
        if result.score_kind != "propensity":
            summary_values["predictive_mp"] = np.nan
            summary_values["predictive_optimal_target_share"] = np.nan

        metric_rows.append(
            {
                "model_name": result.model_name,
                "score_kind": result.score_kind,
                "tuning_metric": result.tuning_metric,
                **summary_values,
            }
        )

        qini_frames.append(evaluation.qini_curve.assign(model_name=result.
            model_name))
        curve_frames.append(evaluation.grid_curve.assign(model_name=result.
            model_name))

        params_payload["models"][result.model_name] = {
            "score_kind": result.score_kind,
            "best_params": _to_builtin(result.best_params),
            "tuning_metric": float(result.tuning_metric),
        }

    summary_metrics = pd.DataFrame(metric_rows).sort_values("mpu", ascending=
        False).reset_index(drop=True)
    qini_curves = pd.concat(qini_frames, ignore_index=True)
    model_curves = pd.concat(curve_frames, ignore_index=True)

    sensitivity_df = pd.DataFrame()
    if not args.skip_sensitivity_sweep:
        sensitivity_df = _build_sensitivity_table(
            model_curves,
            default_basket_value=avg_basket_value,
            default_sms_cost=args.sms_cost,
        )

    scored_validation_path = args.output_dir / "phase2_scored_validation.csv"
    scored_test_path = args.output_dir / "phase2_scored_test.csv"
    summary_metrics_path = args.output_dir / "phase2_summary_metrics.csv"
    qini_curve_path = args.output_dir / "phase2_qini_curves.csv"
    liftup_curve_path = args.output_dir / "phase2_liftup_curves.csv"
    sensitivity_path = args.output_dir / "phase2_profit_sensitivity.csv"
    params_path = args.output_dir / "phase2_model_params.json"

    scored_validation.to_csv(scored_validation_path, index=False)
    scored_test.to_csv(scored_test_path, index=False)
    summary_metrics.to_csv(summary_metrics_path, index=False)
    qini_curves.to_csv(qini_curve_path, index=False)
    model_curves.to_csv(liftup_curve_path, index=False)
    if not sensitivity_df.empty:
        sensitivity_df.to_csv(sensitivity_path, index=False)
```

```
    params_path.write_text(json.dumps(_to_builtin(params_payload), indent=2),
        encoding="utf-8")

    _save_plot_qini(qini_curves, args.output_dir / "qini_curves.png")
    _save_plot_liftup(model_curves, args.output_dir / "liftup_curves.png")
    _save_plot_profit(model_curves, args.output_dir / "profit_vs_target_depth.
        png")
    if not sensitivity_df.empty:
        _save_plot_sensitivity(
            sensitivity_df,
            default_basket_value=avg_basket_value,
            default_sms_cost=args.sms_cost,
            output_path=args.output_dir / "profit_sensitivity.png",
        )

    print("Phase II pipeline completed")
    print("==========================")
    print(f"Validation rows: {len(scored_validation):,}")
    print(f"Test rows:       {len(scored_test):,}")
    print(f"Feature count:   {len(bundle.feature_columns):,}")
    print(f"Avg basket:      {avg_basket_value:,.3f}")
    print(f"SMS cost:        {args.sms_cost:,.3f}")
    print()
```

B Reproduction Commands

Listing 5: Commands used to reproduce the work

```
cd "D:\Business Analytics\retailhero-uplift"
python -m pip install -r .\requirements.txt
python .\verify_phase1_findings.py
python .\phase2_pipeline.py --reuse-features
python .\verify_phase2_outputs.py
```

References

[1] Devriendt, F., Guns, T., and Verbeke, W. (2021). Learning to rank for uplift modeling. In the value-driven uplift modeling literature adapted here for campaign targeting.

[2] Lo, V. S. Y. (2002). The True Lift Model. A treatment-interaction approach to uplift-style modeling.